

Password Managers

It seems like everything requires a password nowadays. We use passwords for our home Wi-Fi, social networks, bank accounts, business emails, and even our favorite applications and websites. According to this [NordPass study](#), the average person has 100 passwords, which is one of the reasons that most people reuse passwords or create simple passwords.

With all this in mind, we need different and secure passwords, but not everyone can memorize hundreds of passwords that meet the complexity required for a password to be secure. We need something that can help us to organize our passwords securely. A [password manager](#) is an application that allows users to store their passwords and secrets in an encrypted database. In addition to keeping our passwords and sensitive data safe, they also have features to generate and manage robust and unique passwords, 2FA, fill web forms, browser integration, synchronization between multiple devices, security alerts, among other features.

How Does a Password Manager Work?

The implementation of password managers varies depending on the manufacturer, but most work with a master password to encrypt the database.

The encryption and authentication work using different [Cryptographic hash functions](#) and [key derivations functions](#), to prevent unauthorized access to our encrypted password database and its content. The way this works depends on the manufacturer and if the password manager is offline or online.

Let's break down the common password managers and how they work.

Online Password Managers

One of the key elements when deciding on a password manager is convenience. A typical person has 3 or 4 devices and uses them to log in to different websites, applications, etc. An online password manager allows the user to synchronize its encrypted password database between multiples devices, most of them provide:

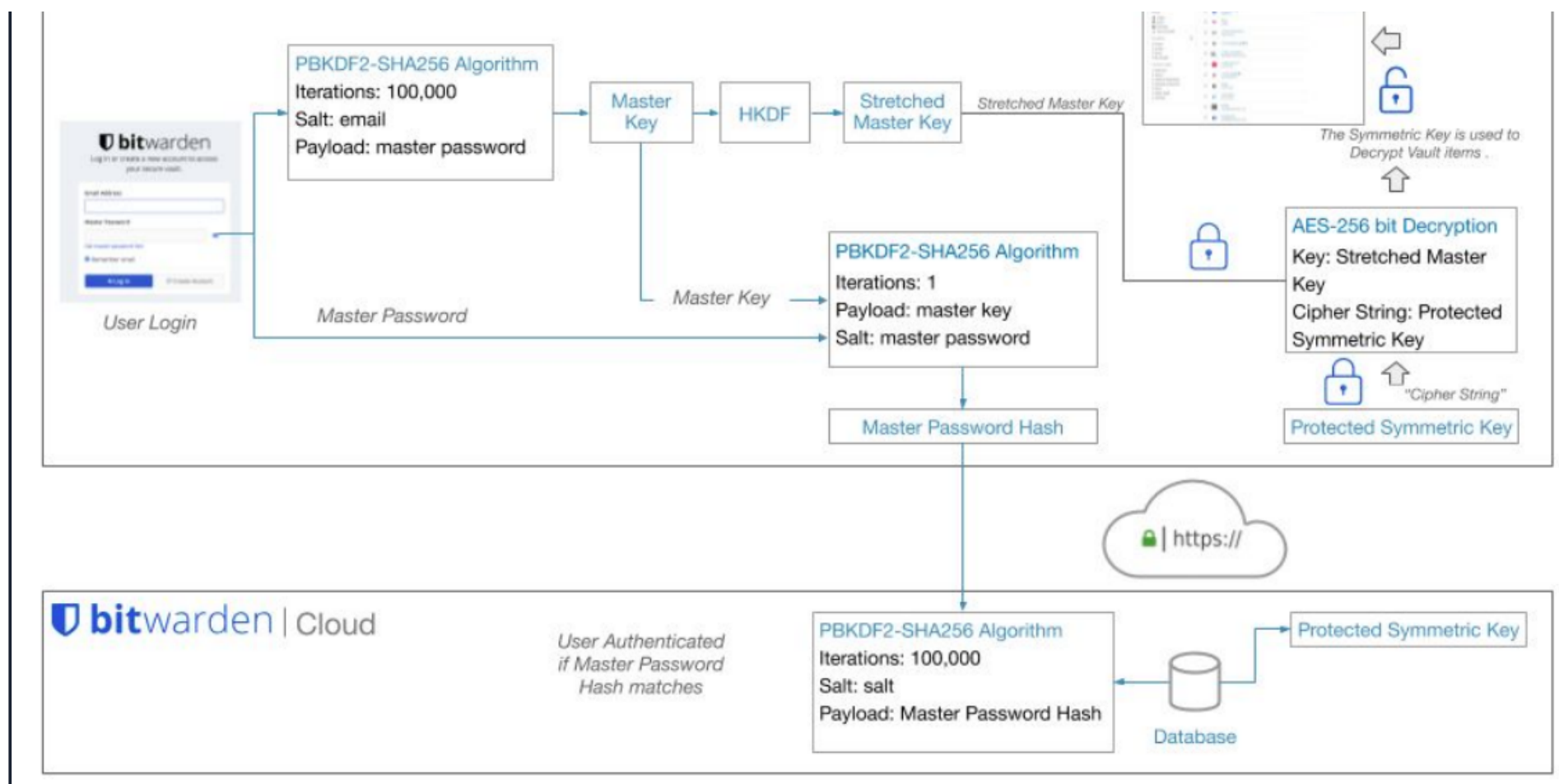
- A mobile application.
- A browser add-on.
- Some other features that we'll discuss later in this section.

All password manager vendors have their way of managing their security implementation, and they usually provide a technical document that describes how it works. You can check [Bitwarden](#), [1Password](#) and [LastPass](#) documentation as a reference, but there are many others. Let's talk about how this generally works.

A common implementation for online password managers is deriving keys based on the master password. Its purpose is to provide a [Zero Knowledge Encryption](#), which means that no one, except you (not even the service provider), can access your secured data. To achieve this, they commonly derive the master password. Let us use Bitwarden's technical implementation for password derivation to explain how it works:

1. Master Key: created by some function to turn the master password into a hash.
2. Master Password Hash: created by some function to turn the master password with a combination of the master key into a hash to authenticate to the cloud.
3. Decryption Key: created by some function using the master key to form a Symmetric Key to Decrypt Vault items.





This is a simple way to illustrate how password managers work, but the common implementation is more complex. You can check the technical documents above or watch the [How Password Managers Work - Computerphile](#) video.

Most popular online password managers are:

1. [1Password](#)
2. [Bitwarden](#)
3. [Dashlane](#)
4. [Keeper](#)
5. [Lastpass](#)
6. [NordPass](#)
7. [RoboForm](#)

Local Password Managers

Some companies and individuals prefer to manage their security for different reasons and not rely on services provided by third parties. Local password managers offer this option by storing the database locally and putting the responsibility on the user to protect their content and the location where it is stored. [Dashlane](#) wrote a blog post [Password Manager Storage: Cloud vs. Local](#) which can help you discover the pros and cons of each storage. The blog post states, "At first it might seem like this makes local storage more secure than cloud storage, but cybersecurity is not a simple discipline." You can use this blog to start your research and understand which method would better serve the different scenarios where you need to manage passwords.

Local password managers encrypt the database file using a master key. The master key can consist of one or multiple components: a master password, a key file, a username, password, etc. Usually, all parts of the master key are required to access the database.

Local password managers' encryption is similar to cloud implementations. The most noticeable difference is data transmission and authentication. To encrypt the database, local password managers focus on securing the local database using different cryptographic hash functions (depending on the manufacturer). They also use the key derivation function (random salt) to avoid precomputing keys and hinder dictionary and guessing attacks. Some offer memory protection and keylogger protection using a secure desktop, similar to Windows User Account Control (UAC).

The most popular local password managers are:

1. [KeePass](#)
2. [KeePassX](#)

2. [KwalleManager](#)
 3. [Pleasant Password Server](#)
 4. [Password Safe](#)
-

Features

Let's imagine we use Linux, Android, and Chrome OS. We access all of our applications and websites from any device. We want to synchronize all passwords and secure notes across all devices. We need extra protection with 2FA, and our budget is 1USD monthly. That information may help us identify the correct password manager for us.

When deciding on a cloud or local password manager, we need to understand its features, [Wikipedia](#) has a list of password managers (online and local) as well as some of their features. Here's a list of the most common features for password managers:

1. [2FA](#) support.
 2. Multi-platform (Android, iOS, Windows, Linux, Mac, etc.).
 3. Browser Extension.
 4. Login Autocomplete.
 5. Import and export capabilities.
 6. Password generation.
-

Alternatives

Passwords are the most common way of authentication but not the only one. As we learn from this module, there are multiple ways to compromise a password, cracking, guessing, shoulder surfing, etc., but what if we don't need a password to log in? Is such a thing possible?

By default, most operating systems and applications do not support any alternative to a password. Still, administrators can use 3rd party identity providers or applications to configure or enhance identity protection across their organizations. Some of the most common ways to secure identities beyond passwords are:

1. [Multi-factor Authentication](#).
 2. [FIDO2](#) open authentication standard, which enables users to leverage common devices like [Yubikey](#), to authenticate easily. For a more extended device list, you can see [Microsoft FIDO2 security key providers](#).
 3. [One-Time Password \(OTP\)](#).
 4. [Time-based one-time password \(TOTP\)](#).
 5. [IP restriction](#).
 6. Device Compliance. Examples: [Endpoint Manager](#) or [Workspace ONE](#)
-

Passwordless

Multiples companies like [Microsoft](#), [Auth0](#), [Okta](#), [Ping Identity](#), etc, are trying to promote the [Passwordless](#) strategy, to remove the password as the way of authentication.

[Passwordless](#) authentication is achieved when an authentication factor other than a password is used. A password is a knowledge factor, meaning it's something a user knows. The problem with relying on a knowledge factor alone is that it's vulnerable to theft, sharing, repeat use, misuse, and other risks. Passwordless authentication ultimately means no more passwords. Instead, it relies on a possession factor, something a user has, or an inherent factor, which a user is, to verify user identity with greater assurance.

As new technology and standards evolve, we need to investigate and understand the details of its implementation to understand if those alternatives will or not provide the security we need for the authentication process. You can read more about Passwordless

authentication and different vendors' strategies:

1. [Microsoft Passwordless](#)
2. [Auth0 Passwordless](#)
3. [Okta Passwordless](#)
4. [PingIdentity](#)

There are many options when it comes to protecting passwords. Choosing the right one will come down to the individual or company's requirements. It is common for people and companies to use different password protection methods for various purposes.

← Previous

Next →

 Cheat Sheet

 Resources

Table of Contents

Introduction

Theory of Protection	✓
Credential Storage	✓
John The Ripper	✓

Remote Password Attacks

 Network Services	✓
 Password Mutations	✓
 Password Reuse / Default Passwords	✓

Windows Local Password Attacks

 Attacking SAM	✓
 Attacking LSASS	✓
 Attacking Active Directory & NTDS.dit	✓
 Credential Hunting in Windows	✓

Linux Local Password Attacks

 Credential Hunting in Linux	✓
 Passwd, Shadow & Opasswd	✓

Windows Lateral Movement

 Pass the Hash (PtH)	✓
 Pass the Ticket (PtT) from Windows	✓
 Pass the Ticket (PtT) from Linux	✓

Cracking Files

 Protected Files	✓
 Protected Archives	✓

Password Management

Password Policies

[Password Managers](#)

Skills Assessment

 Password Attacks Lab - Easy

 Password Attacks Lab - Medium

 Password Attacks Lab - Hard

My Workstation

OFFLINE

 Start Instance

 / 1 spawns left